Logistic Regression

Regression Tasks

Regression Tasks

A

```
double ml_model(A input){
    ....
    return output;
}
```

B (double)

Data Set

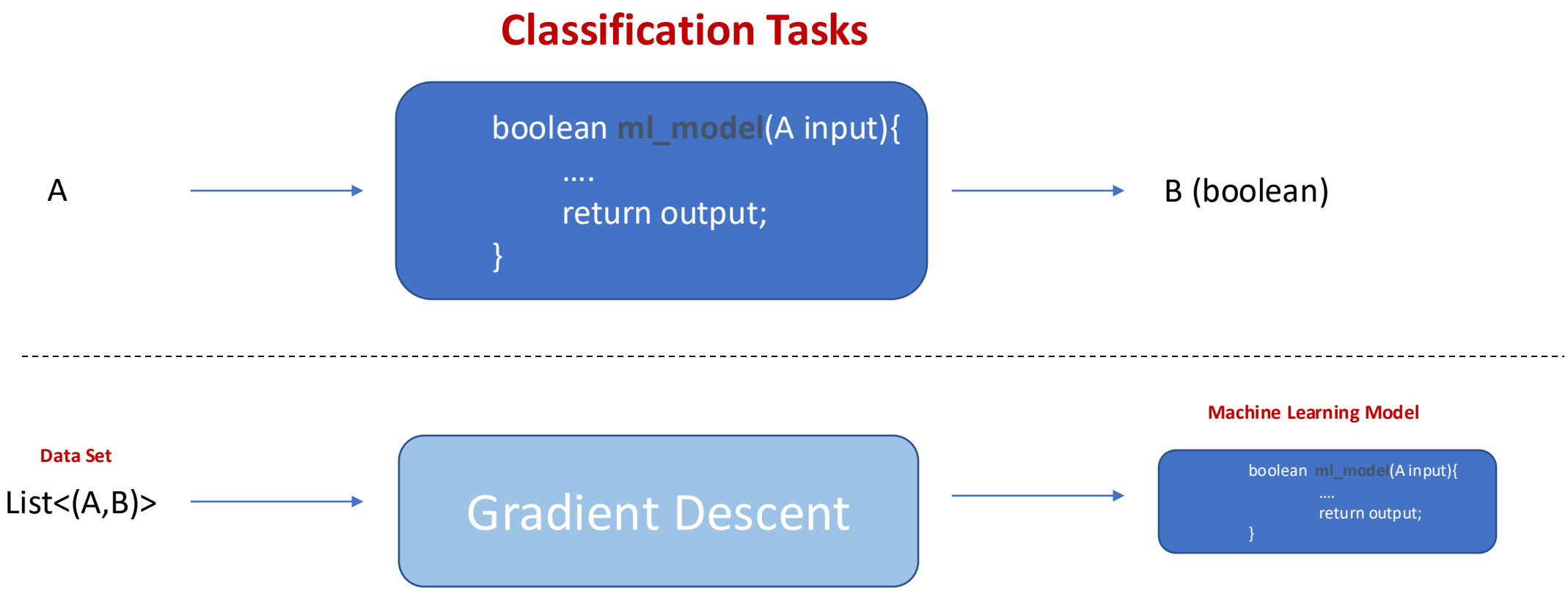
List<(A,B)>

Gradient Descent

Machine Learning Model

```
Double ml_model(A input){
    ....
    return output;
}
```

Classification Tasks

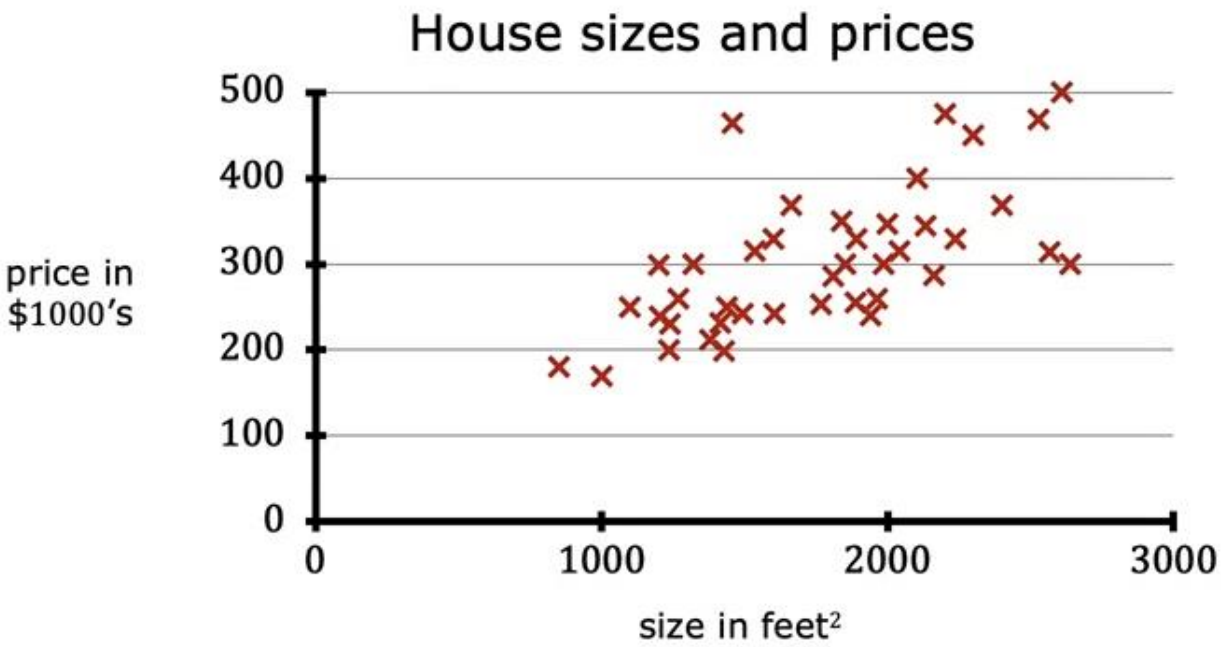


Classification Tasks

Question	Answer "y"
Is this email spam?	
Is the transaction fraudulent?	
Is the tumor malignant?	

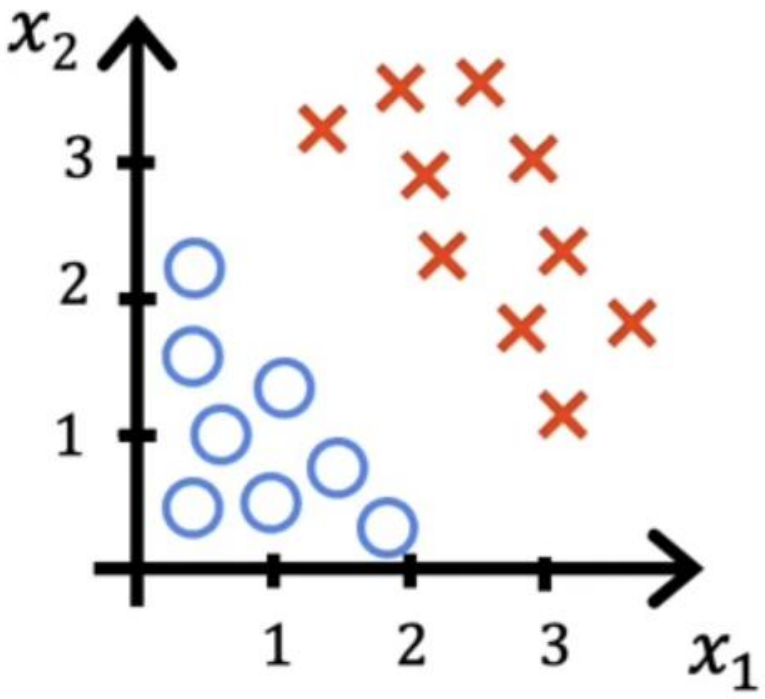
Linear Regression

$$f_{\vec{w},b}(\vec{x}) = w_1 x_1 + \cdots + w_n x_n + b$$



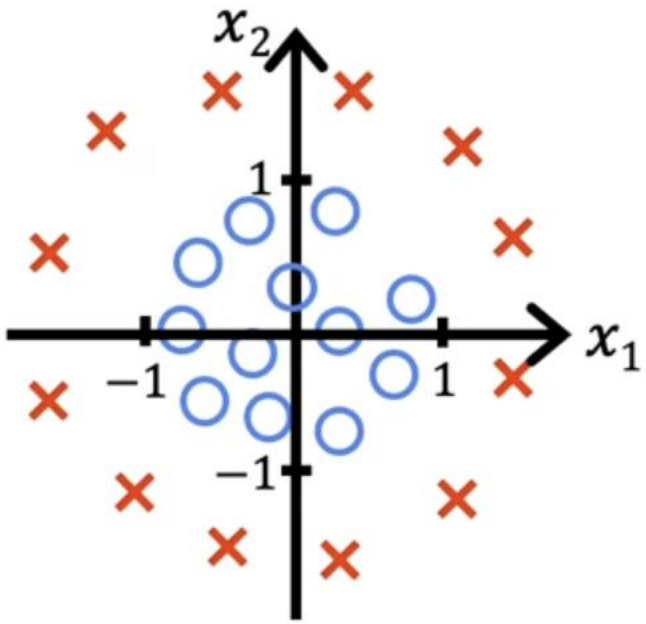
Decsision Boundary

$$f_{\vec{w},b}(\vec{x}) = g(z) = g(w_1 x_1 + w_2 x_2 + b)$$



Non Linear Decsision Boundary

$$f_{\vec{w},b}(\vec{x}) = g(z) =$$



Exercise 1: Sigmoid Function

Exercise 1

Please complete the `sigmoid` function to calculate

$$g(z) = \frac{1}{1 + e^{-z}}$$

Note that

- `z` is not always a single number, but can also be an array of numbers.
- If the input is an array of numbers, we'd like to apply the sigmoid function to each value in the input array.

If you get stuck, you can check out the hints presented after the cell below to help you with the implementation.

Cost Funtion Logistic Regression

Training set

tumor size (cm)	...	patient's age	malignant?
10		52	1
2		73	0
5		55	0
12		49	1
...		...	...

$$f_{\vec{w},b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w}\cdot\vec{x}+b)}}$$

Cost Funtion Logistic Regression

$$J(\vec{\mathrm{w}}, b) = \frac{1}{m}\sum_{i=1}^{m} L\left(f_{\vec{\mathrm{w}},b}\left(\vec{\mathrm{x}}^{(i)}\right), y^{(i)}\right)$$

$$L\left(f_{\vec{\mathrm{w}},b}\left(\vec{\mathrm{x}}^{(i)}\right), y^{(i)}\right) = \begin{cases} -\log\left(f_{\vec{\mathrm{w}},b}\left(\vec{\mathrm{x}}^{(i)}\right)\right) & \text{if } y^{(i)} = 1 \\ -\log\left(1 - f_{\vec{\mathrm{w}},b}\left(\vec{\mathrm{x}}^{(i)}\right)\right) & \text{if } y^{(i)} = 0 \end{cases}$$

Cost Funtion Logistic Regression

$$J(\vec{w}, b) = \frac{1}{m}\sum_{i=1}^{m} L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right)$$

$$L\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right), y^{(i)}\right) = \begin{cases} -\log\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right) & \text{if } y^{(i)} = 1 \\ -\log\left(1 - f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right) & \text{if } y^{(i)} = 0 \end{cases}$$

$$J(\vec{w}, b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right) + \left(1 - y^{(i)}\right)\log\left(1 - f_{\vec{w},b}\left(\vec{x}^{(i)}\right)\right)\right]$$

Gradient Descent Logistic Regression

$$J(\vec{\mathrm{w}}, b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(f_{\vec{\mathrm{w}},b}\left(\vec{\mathrm{x}}^{(i)}\right)\right) + \left(1 - y^{(i)}\right)\log\left(1 - f_{\vec{\mathrm{w}},b}\left(\vec{\mathrm{x}}^{(i)}\right)\right)\right]$$

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{\mathrm{w}}, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{\mathrm{w}}, b)$$

}

$$\frac{\partial}{\partial w_j} J(\vec{\mathrm{w}}, b) = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{\mathrm{w}},b}\left(\vec{\mathrm{x}}^{(i)}\right) - y^{(i)}\right)x_j^{(i)}$$

$$\frac{\partial}{\partial b} J(\vec{\mathrm{w}}, b) = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{\mathrm{w}},b}\left(\vec{\mathrm{x}}^{(i)}\right) - y^{(i)}\right)$$

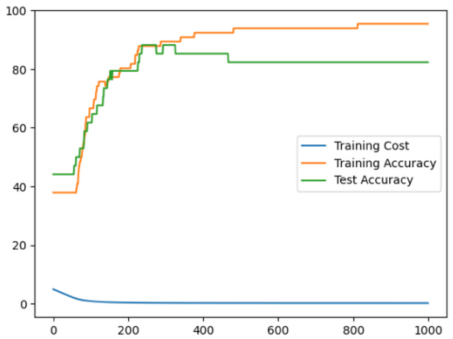
Exercise 2: Learn a Logistic Regression

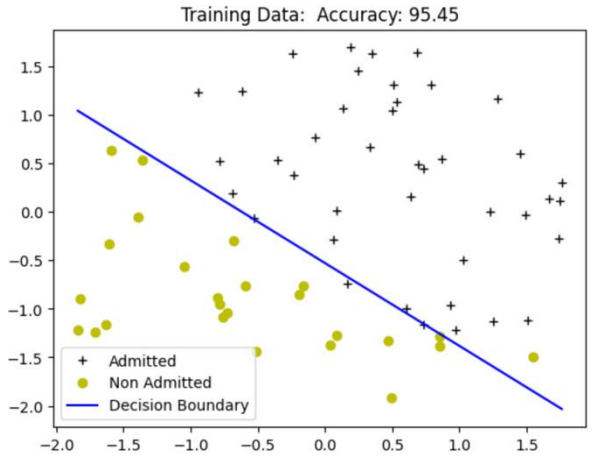
Exercise 2: Learn the Logistic Regression Model

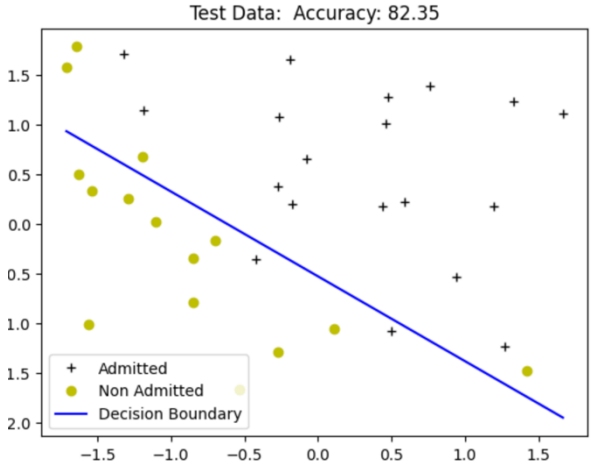
We will now use the final parameters from gradient descent to plot the linear fit.

```
np.random.seed(1)
intial_w = 0.01 * (np.random.rand(2).reshape(-1,1) - 0.5)
initial_b = -8


# Some gradient descent settings
iterations = 1000
alpha = 0.1

w,b, J_history_train, J_history_test = gradient_descent_val(X_train ,y_train, initial_w, initial_b,
                                    compute_cost, compute_gradient, alpha, iterations, 0)
```

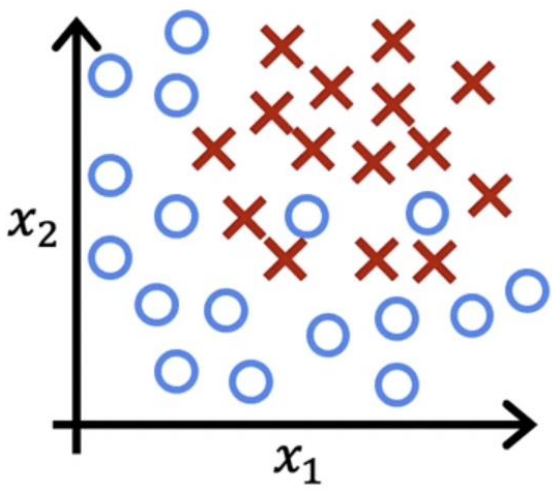
```
Iteration    0: Training Cost     4.91, Train Accuracy    37.88, Test Accuracy    44.12
Iteration   20: Training Cost     3.81, Train Accuracy    37.88, Test Accuracy    44.12
Iteration   40: Training Cost     2.73, Train Accuracy    37.88, Test Accuracy    44.12
Iteration   60: Training Cost     1.78, Train Accuracy    37.88, Test Accuracy    47.06
Iteration   80: Training Cost     1.17, Train Accuracy    54.55, Test Accuracy    52.94
Iteration  100: Training Cost     0.84, Train Accuracy    66.67, Test Accuracy    61.76
Iteration  120: Training Cost     0.64, Train Accuracy    74.24, Test Accuracy    67.65
Iteration  140: Training Cost     0.51, Train Accuracy    74.24, Test Accuracy    73.53
Iteration  160: Training Cost     0.42, Train Accuracy    77.27, Test Accuracy    79.41
Iteration  180: Training Cost     0.36, Train Accuracy    80.30, Test Accuracy    79.41
Iteration  200: Training Cost     0.32, Train Accuracy    80.30, Test Accuracy    79.41
```

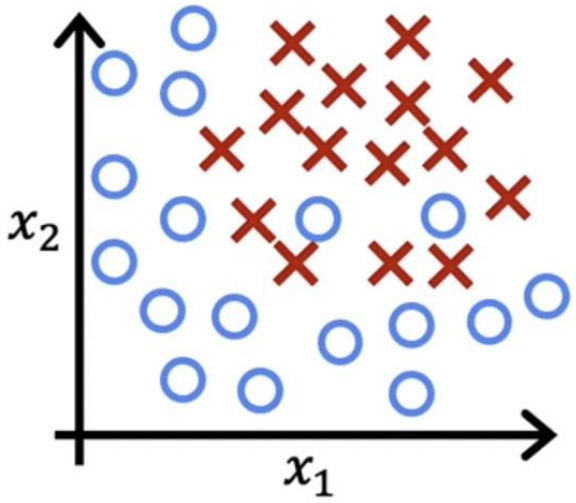


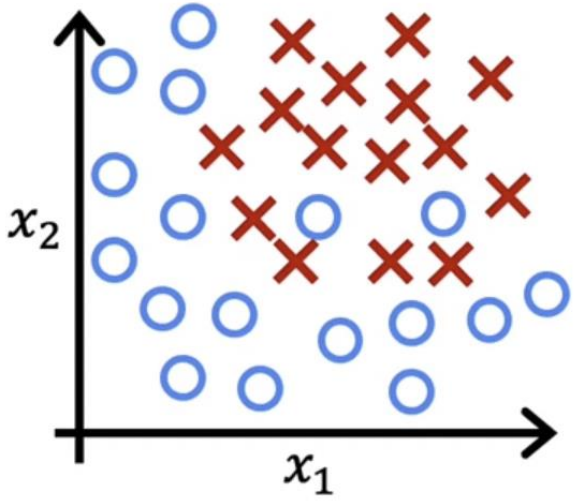




OverFitting

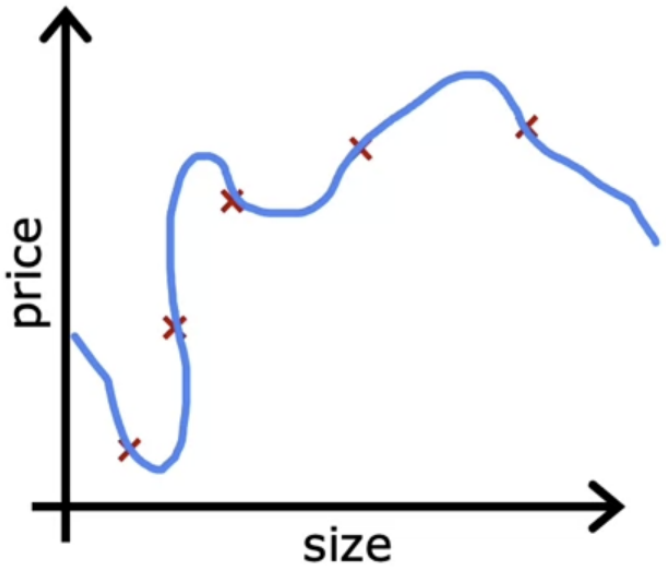






Addressing Overfitting

Collect more training examples



Addressing Overfitting

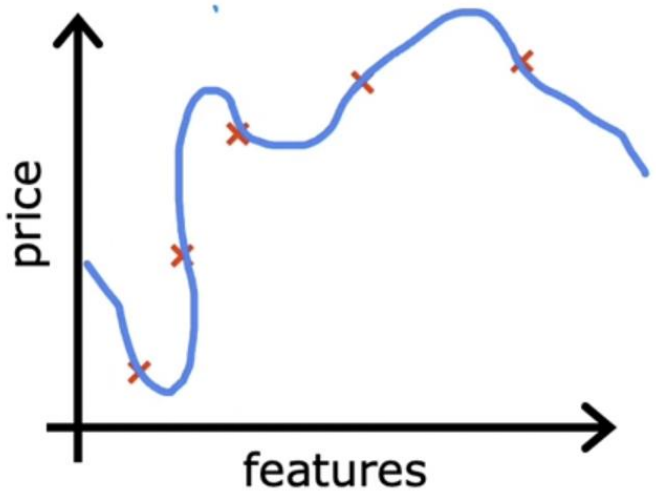
Select features to include/exclude

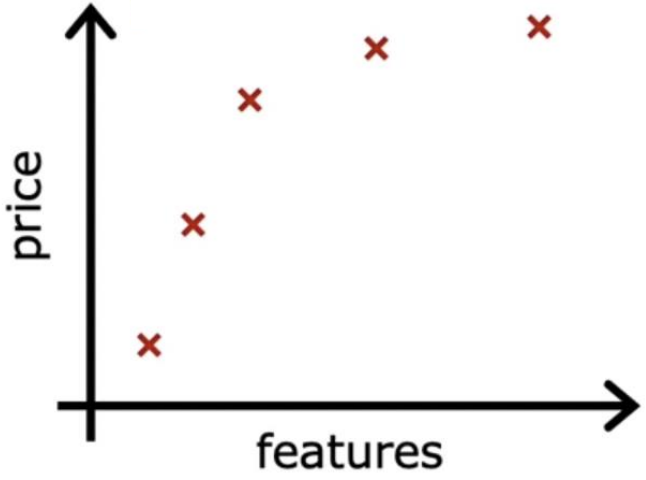
size	bedrooms	floors	age	avg income	...	distance to coffee shop	price

Addressing Overfitting

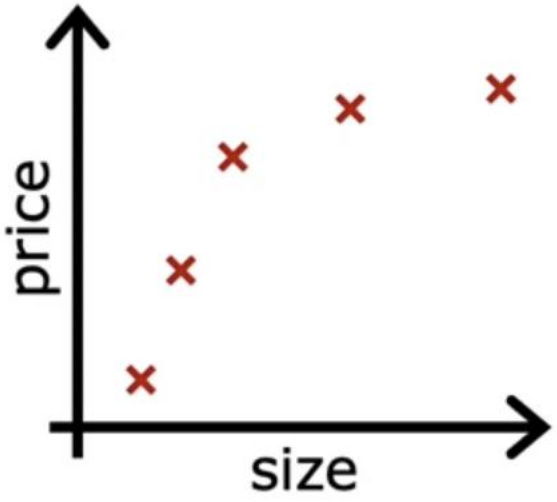
Regularization

Reduce the size of parameters w_j

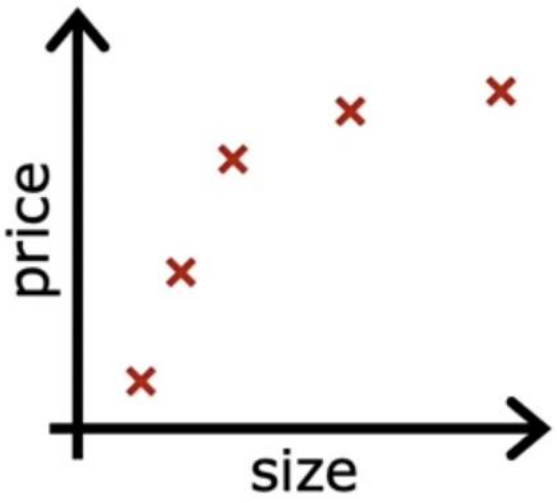




Cost Funtion with Regularization



$$w_1 x + w_2 x^2 + b$$

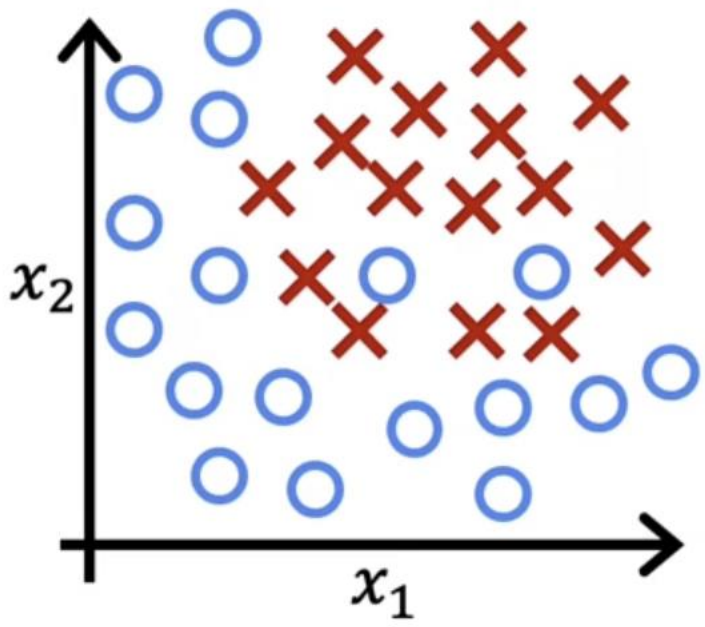


$$w_1 x + w_2 x^2 + w_3 x^3 + w_4 x^4 + b$$

$$\min_{\vec{w},b} \frac{1}{2m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right)^2 + \boxed{\frac{\lambda}{2m} \sum_{j=0}^{n-1} w_j^2}$$

Regularization Term

Logistic Regression with Regularization



$$J(\vec{\mathrm{w}}, b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(f_{\vec{\mathrm{w}},b}(\vec{\mathrm{x}}^{(i)})\right) + (1 - y^{(i)})\log\left(1 - f_{\vec{\mathrm{w}},b}(\vec{\mathrm{x}}^{(i)})\right)\right] + \boxed{\frac{\lambda}{2m}\sum_{j=0}^{n-1} w_j^2}$$

Regularization Term

Exercise 3: Compute Regularized Cost

3 - Regularized Logistic Regression

In this part of the exercise, you will implement regularized logistic regression

While the feature mapping allows us to build a more expressive classifier, it is also more susceptible to overfitting. In the next parts of the exercise, you will implement regularized logistic regression to fit the data and also see for yourself how regularization can help combat the overfitting problem.

3.1 Cost function for regularized logistic regression

In this part, you will implement the cost function for regularized logistic regression.

Recall that for regularized logistic regression, the cost function is of the form

$$J(\mathbf{w},b) = \frac{1}{m} \sum_{i=0}^{m-1} \left[-y^{(i)} \log\left(f_{\mathbf{w},b}\left(\mathbf{x}^{(i)} \right) \right) - \left(1 - y^{(i)}\right) \log \left(1 - f_{\mathbf{w},b}\left(\mathbf{x}^{(i)} \right) \right) \right] + \frac{\lambda}{2m} \sum_{j=0}^{n-1} w_j^2$$

Compare this to the cost function without regularization (which you implemented above), which is of the form

$$J(\mathbf{w}.b) = \frac{1}{m}\sum_{i=0}^{m-1} \left[(-y^{(i)} \log\left(f_{\mathbf{w},b}\left(\mathbf{x}^{(i)} \right) \right) - \left(1 - y^{(i)}\right) \log \left(1 - f_{\mathbf{w},b}\left(\mathbf{x}^{(i)} \right) \right)\right]$$

The difference is the regularization term, which is

$$\frac{\lambda}{2m} \sum_{j=0}^{n-1} w_j^2$$

Note that the b parameter is not regularized.

```python
# UNQ_C5
def compute_cost_reg(X, y, w, b, lambda_ = 1):
    """
    Computes the cost over all examples
    Args:
      X : (array_like Shape (m,n)) data, m examples by n features
      y : (array_like Shape (m,)) target value
      w : (array_like Shape (n,)) Values of parameters of the model
      b : (array_like Shape (n,)) Values of bias parameter of the model
      lambda_ : (scalar, float)    Controls amount of regularization
    Returns:
      total_cost: (scalar)         cost
    """

    m, n = X.shape

    # Calls the compute_cost function that you implemented above
    cost_without_reg = compute_cost(X, y, w, b)

    ### START CODE HERE ###

    # You need to calculate this value
    reg_cost =  0

    ### END CODE HERE ###

    # Add the regularization cost to get the total cost
    total_cost = cost_without_reg + reg_cost

    return total_cost
```

Gradient Descent witougt Regularization

$$J(\vec{\mathrm{w}}, b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(f_{\vec{\mathrm{w}},b}\left(\vec{\mathrm{x}}^{(i)}\right)\right) + \left(1 - y^{(i)}\right)\log\left(1 - f_{\vec{\mathrm{w}},b}\left(\vec{\mathrm{x}}^{(i)}\right)\right)\right]$$

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{\mathrm{w}}, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{\mathrm{w}}, b)$$

}

$$\frac{\partial}{\partial w_j} J(\vec{\mathrm{w}}, b) = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{\mathrm{w}},b}\left(\vec{\mathrm{x}}^{(i)}\right) - y^{(i)}\right)x_j^{(i)}$$

$$\frac{\partial}{\partial b} J(\vec{\mathrm{w}}, b) = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{\mathrm{w}},b}\left(\vec{\mathrm{x}}^{(i)}\right) - y^{(i)}\right)$$

Gradient Descent with Regularization

$$J(\vec{\mathrm{w}}, b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(f_{\vec{\mathrm{w}},b}(\vec{\mathrm{x}}^{(i)})\right) + (1 - y^{(i)})\log\left(1 - f_{\vec{\mathrm{w}},b}(\vec{\mathrm{x}}^{(i)})\right) + \boxed{\frac{\lambda}{2m}\sum_{j=0}^{n-1} w_j^2}\right.$$

Regularization Term

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{\mathrm{w}}, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{\mathrm{w}}, b)$$

}

$$\frac{\partial}{\partial w_j} J(\vec{\mathrm{w}}, b) = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{\mathrm{w}},b}(\vec{\mathrm{x}}^{(i)}) - y^{(i)}\right)x_j^{(i)} + \boxed{\frac{\lambda}{m} w_j}$$

Regularization Term

$$\frac{\partial}{\partial b} J(\vec{\mathrm{w}}, b) = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{\mathrm{w}},b}(\vec{\mathrm{x}}^{(i)}) - y^{(i)}\right)$$

Exercise 4: Regularized Gradient

3.2 Gradient for regularized logistic regression

In this section, you will implement the gradient for regularized logistic regression.

The gradient of the regularized cost function has two components. The first, $\frac{\partial J(\mathbf{w},b)}{\partial b}$ is a scalar, the other is a vector with the same shape as the parameters $\mathbf{w}$, where the jth element is defined as follows:

$$\frac{\partial J(\mathbf{w},b)}{\partial b} = \frac{1}{m}\sum_{i=0}^{m-1}(f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)})$$

$$\frac{\partial J(\mathbf{w},b)}{\partial w_j} = \left(\frac{1}{m}\sum_{i=0}^{m-1}(f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)})x_j^{(i)}\right) + \frac{\lambda}{m}w_j \quad \text{for } j = 0...(n-1)$$

Compare this to the gradient of the cost function without regularization (which you implemented above), which is of the form

$$\frac{\partial J(\mathbf{w},b)}{\partial b} = \frac{1}{m}\sum_{i=0}^{m-1}(f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - \mathbf{y}^{(i)}) \tag{2}$$

$$\frac{\partial J(\mathbf{w},b)}{\partial w_j} = \frac{1}{m}\sum_{i=0}^{m-1}(f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - \mathbf{y}^{(i)})x_j^{(i)} \tag{3}$$

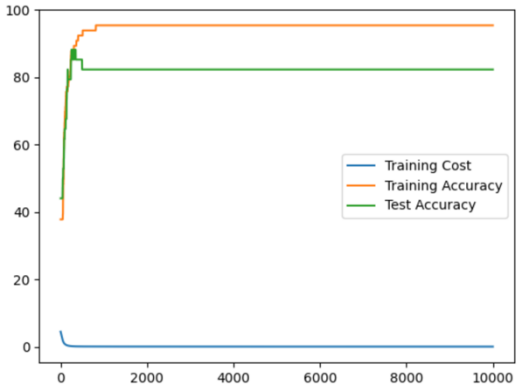
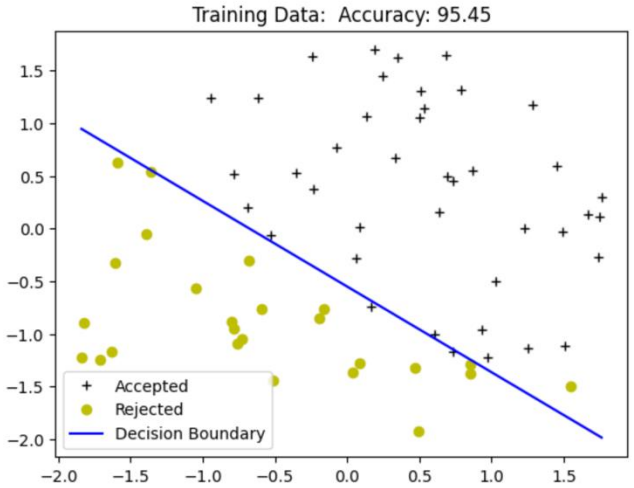
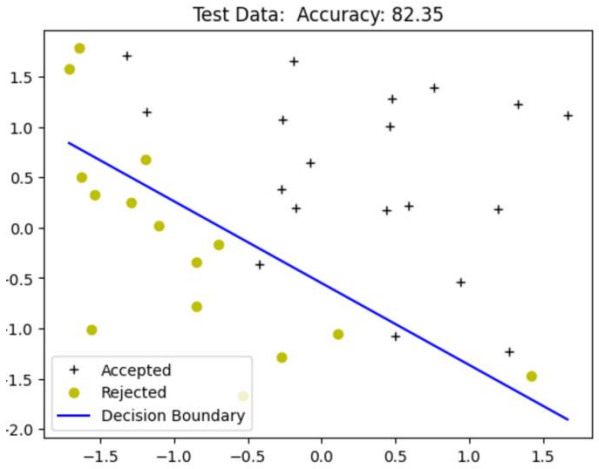
As you can see, $\frac{\partial J(\mathbf{w},b)}{\partial b}$ is the same, the difference is the following term in $\frac{\partial J(\mathbf{w},b)}{\partial w}$, which is

$$\frac{\lambda}{m}w_j \quad \text{for } j = 0...(n-1)$$

```python
# UNQ_C6
def compute_gradient_reg(X, y, w, b, lambda_ = 1):
    """
    Computes the gradient for linear regression

    Args:
      X : (ndarray Shape (m,n))   variable such as house size
      y : (ndarray Shape (m,))    actual value
      w : (ndarray Shape (n,))    values of parameters of the model
      b : (scalar)                value of parameter of the model
      lambda_ : (scalar,float)    regularization constant
    Returns
      dj_db: (scalar)             The gradient of the cost w.r.t. the parameter b.
      dj_dw: (ndarray Shape (n,)) The gradient of the cost w.r.t. the parameters w.

    """
    m, n = X.shape

    dj_db, dj_dw = compute_gradient(X, y, w, b)

    ### START CODE HERE ###

    dj_dw += 0

    ### END CODE HERE ###

    return dj_db, dj_dw
```

Exercise 5: Regularized Logistic Regression

Exercise 5: Learning Regularized Logistic Regression

- Train the model for each lambda parameter.
- See how changing lambda affects the decision boundary.
- See how changing lambda affects the classification accuracy.

```
np.random.seed(0)
initial_w = np.random.rand(X_train.shape[1])
initial_b = -8

lambda_=0
# Some gradient descent settings
iterations = 10000
alpha = 0.1

w,b, J_history,_ = gradient_descent_val(X_train, y_train, initial_w, initial_b,
                                        compute_cost_reg, compute_gradient_reg,
                                        alpha, iterations, lambda_)
```